

Web-Architekturen

Web-Engineering II

Prof. Dr. Sebastian von Klinski

Einleitung

Architektur

- Wozu ist Architektur für eine Web-Anwendung wichtig?
- Architektur ist...

„... Übersicht über die Komponenten eines Gesamtsystems“

„... dient dem Verständnis des Systems als Ganzes“

„Was für ein Haus würde man bekommen, wenn man einfach loslegt und die Wände zieht, ohne sich vorher Gedanken über Statik, Grundriss, Strom- und Wasserleitung Gedanken zu machen?“

Architektur

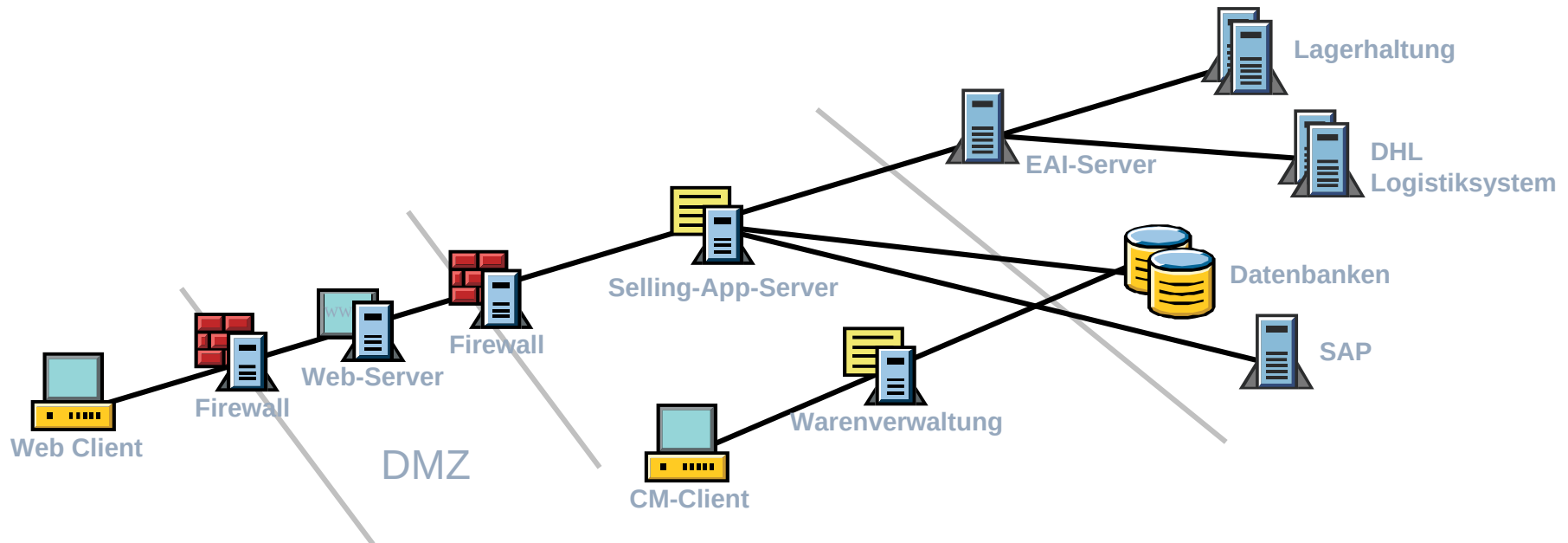
- Jede Architektur bring spezifische Vor- und Nachteile mit sich
 - Wurde falsche Architektur gewählt, ...
 - Schwierige Umsetzung/ Erweiterung des Systems
 - Steigt häufig der Arbeitsaufwand
 - Können einige Probleme (Performance, Benutzerfreundlichkeit, Erweiterbarkeit, etc.) in der Regel nicht mehr behoben werden
 - Nicht selten scheitert das IT-Projekt
 - Gute Architektur → langfristige Nutzung des IT-Systems möglich
- Vor der Umsetzung richtige Architektur festlegen
- Software-Architektur
 - Anwendungsarchitektur
 - Systemarchitektur

Software-Architektur

- **Softwarearchitektur** beschreibt die wesentlichen Komponenten einer Software-Anwendung und deren Zusammenspiel mit anderen Komponenten
- Relevante Aspekte sind Design von Klassen und Schichten, Einsatz von Patterns, etc.
- Patterns: Bewährte Lösungsansätze für eine gute Software-Architektur
 - **Design-Patterns:** z.B. Singleton, Factory, Observer, Composite, Iterator, Decorator, Facade, Proxy,
 - **J2EE-Patterns:** Front Controller, Data Access Object, Session Facade, Transfer Object
 - **Layer-Design:** Service-Layer, DAO-Layer, Persistence-Layer, etc.
- Design abhängig von Sprache (Java, JavaScript, C#, etc.) und Laufzeitumgebung (Node.js, Spring, J2EE, etc.)

Anwendungsarchitektur

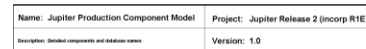
- **Anwendungsarchitektur** beschreibt die wesentlichen Komponenten einer Software-Anwendung



Anwendungsarchitektur

- Wesentliche Fragestellungen
 - Was ist das führende System für welche Daten?
 - Welche Schnittstellen gibt es?
 - Wie sind die Datenflüsse?
 - Welche Anwendungsprozesse gibt es?
 - Etc.
- Maßgeblicher Einfluss auf
 - Performance
 - Ausfallsicherheit
 - Bottlenecks
 - Gestaltungsspielraum in der Zukunft (funktional, Nutzerzahlen, etc.)

- **Systemarchitektur:** Zusammenspiel aller Komponenten einer Systemlandschaft, die zahlreiche IT-Systeme umfasst. Beschreibung der Hardware und / oder Software-Aspekte (z.B. inklusive Schnittstellen)



Architekturmodelle

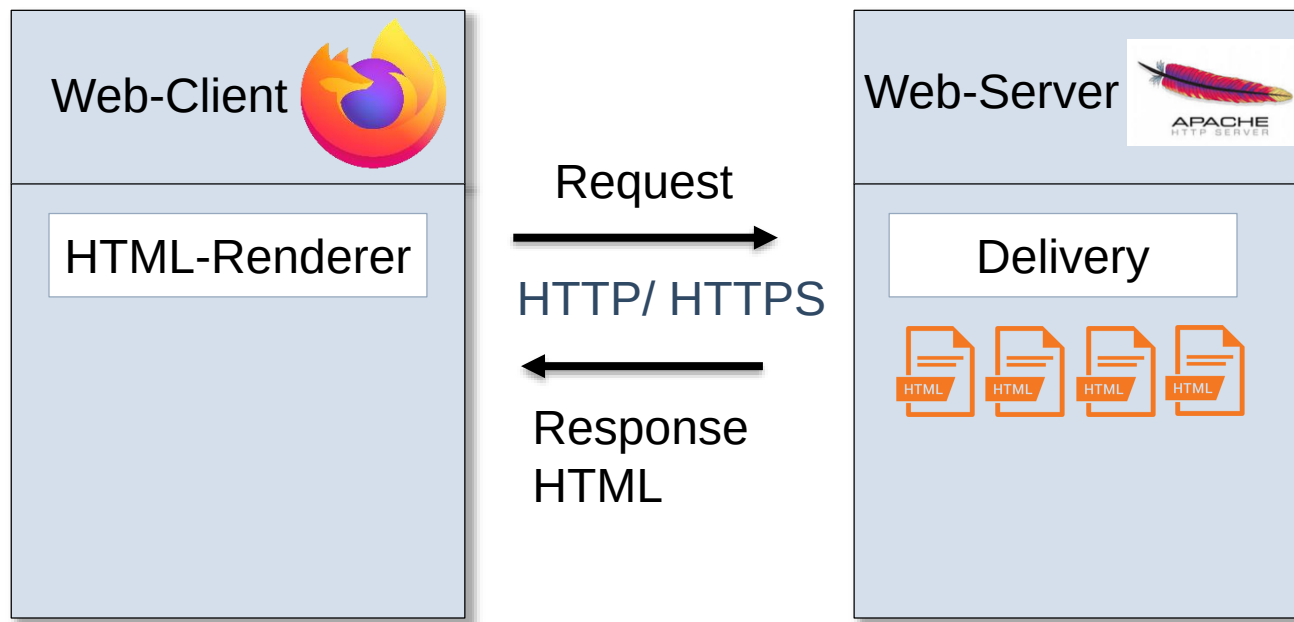
Stand-Alone Applikation

- Anwendung, die ohne weitere IT-Komponenten auskommt
- Beispiele
 - Office-Applikationen (Word, etc.)
 - Graphik-Programme (Photoshop), etc.
- Abhängigkeiten
 - Betriebssystem
- Weitere Aspekte
 - Häufig Update-Mechanismen → Applikation ggfls. dann „Rich Client“



Einfache Web-Architektur: Thin-Client, statische Web-Seiten

- Client: Browser wie Firefox, Chrome, etc.
- HTML-Seiten von Web-Server geliefert



Grundlagen

Browser

- Computerprogramme zur Darstellung von Webseiten im World Wide Web

HTML-Renderer

- Visualisierung von Hypertext Markup Language und Scalable Vector Graphics im Browser

Web-Server

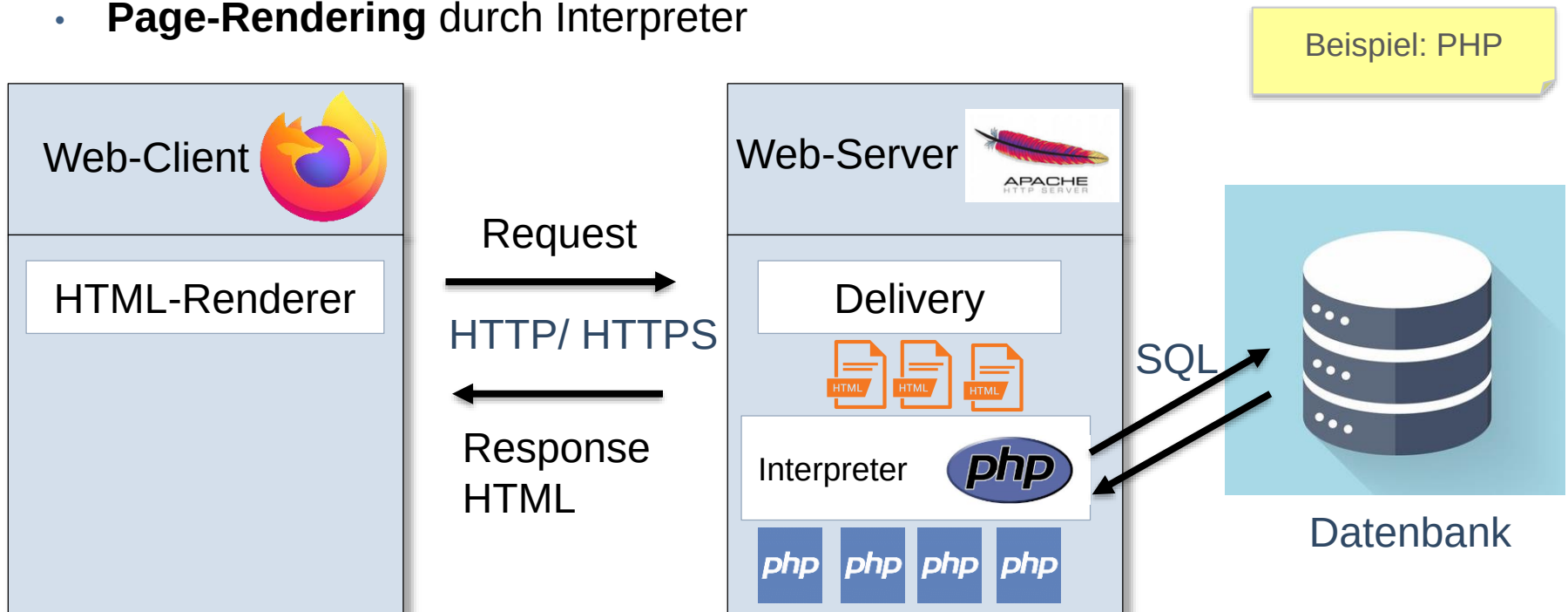
- Server, der Dokumente an Clients wie z. B. Webbrowser überträgt
- Statischer Content (HTML-Seiten, Bilder, CSS, JavaScript)
- Protokolle HTTP (Port 80), HTTPS (Port 443)

Grundlagen

- Was sind Front-end und Back-end?
 - Front-End: beim Nutzer
 - Back-End: beim „System“
- Bei Web-Anwendung...
 - Front-End: ausgeführt im Browser beim Nutzer
 - Back-End: ausgeführt auf dem Server
- Client
 - Nutzt Dienste eines Servers
- Server
 - Bietet Dienste für andere Anwendungen ab
- Ein Server kann gleichzeitig auch Client sein
 - Beispiel: Web-Anwendung greift auf Rest-Service zu
 - Rest-Service nutzt weitere Rest-Services von Drittanbietern
 - Rest-Service ist gleichzeitig Server (für Web-Anwendung) und Client von Drittanbieter-Services

Web-Architektur: Thin-Client, dynamische Web-Seiten

- Unterschiedlichste Ansätze, um dynamische Web-Seiten umzusetzen
- Unterscheidung zwischen serverseitig und clientseitig
- Dynamische Seiten serverseitig: Erstellung von Web-Seite bei Aufruf
- Verwendung von Skriptsprachen: ASP, PHP, JSP, etc.
- **Page-Rendering** durch Interpreter



Grundlagen

Interpreter

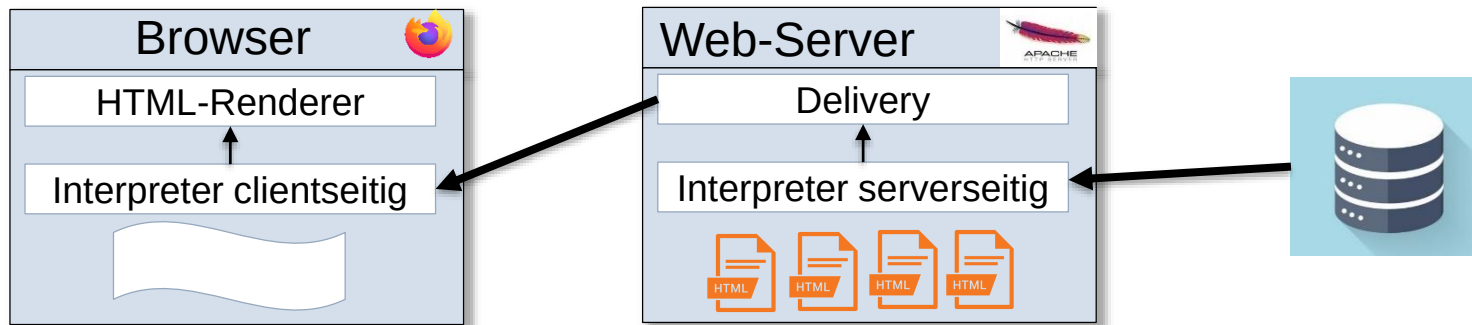
- Interpretiert Tags der Skriptsprache
- Ersetzt Tags durch HTML-Code

Skriptsprachen

- Unterscheidung in Ort, wo der Interpreter läuft (Webserver/ Browser)
- Serverseitig: PHP, ASP, JSP, Python, Perl
- Clientseitig: JavaScript, CoffeeScript, TypeScript

Anmerkungen

- CoffeeScript und TypeScript werden nach JavaScript transkompiliert
- Bei Nodejs (Server) wird JavaScript serverseitig verwendet



Grundlagen: Skriptsprachen

- Programmiersprachen, die über einen Interpreter ausgeführt werden
- Zeilenweises Parsen, Kompilieren und Ausführen zur Laufzeit
- Häufig schwache/ dynamische Typisierung
- Fehler häufig erst zur Laufzeit offenkundig
- Meist schnellere und dynamischere Entwicklung als bei klassischen, kompilierten Sprachen

Arten von Web-Anwendungen: SPA und MPA

Ausprägungen von Web-Anwendungen

- Zahlreiche Varianten je nach Anwendungsgebiet und Anforderungen
- **Single-Page Application (SPA)/ Rich-Web-Client/ Rich Internet Application**
 - Web-Anwendung bestehend aus einer Seite, deren Inhalte dynamisch aktualisiert werden
 - Look-and-Feel wie Desktop-Applikation
 - Einsatz bei hohen Nutzerzahlen aber einfacher Anwendungsdomäne
- **eBusiness-Web-Architekturen**
 - Einsatz bei komplexer Geschäftslogik mit Bedarf für Ausfallsicherheit und Recovery
 - Anwendungslogik in Programmiersprachen wie Java oder C#
 - Beispiel: J2EE-Web-Stack, .Net-Plattform
- **Micro-Service Architektur**
 - Parallelisierung von Entwicklungsarbeit über viele Entwickler
- **Weitere**
 - Message-oriented Architektur, etc.

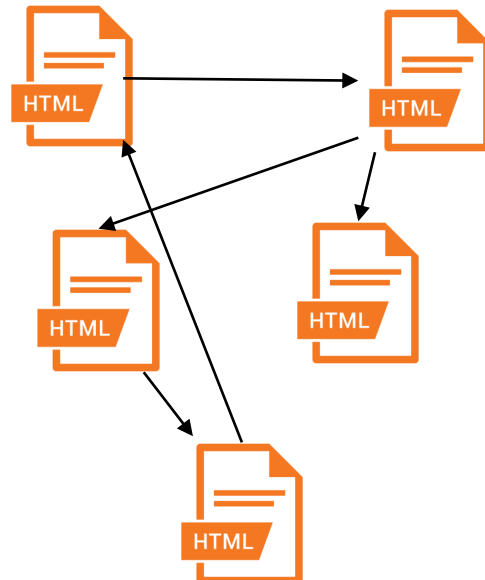
Ausprägungen von Web-Anwendungen

- Fokus in dieser Vorlesung auf Single-Page-Application (SPA)
- Derzeit sehr verbreitet
- Populäre Web-Bibliotheken für diesen Anwendungsbereich (React, TypeScript, Vue, Angular etc.)
- In Zukunft großer Marktanteil bei Web-Anwendungen

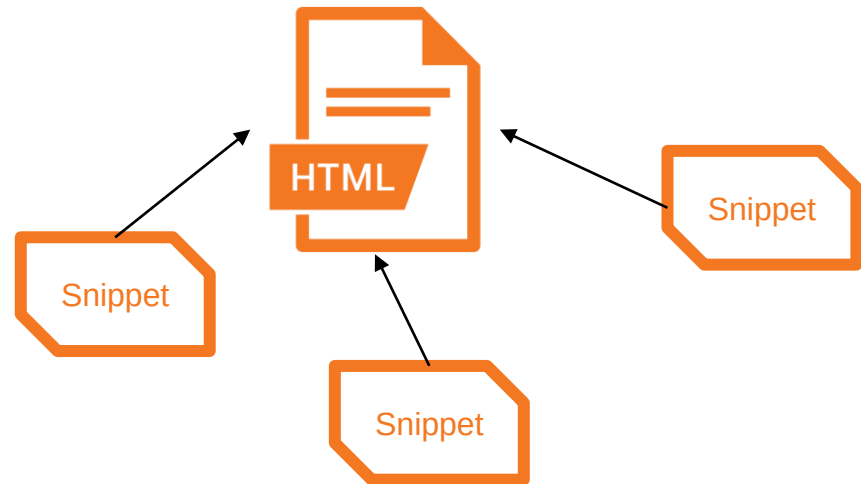
Single-Page-Anwendung

...ist eine Web-Anwendung, welche aus einer einzigen HTML-Seite besteht.

Multi-Page Anwendung



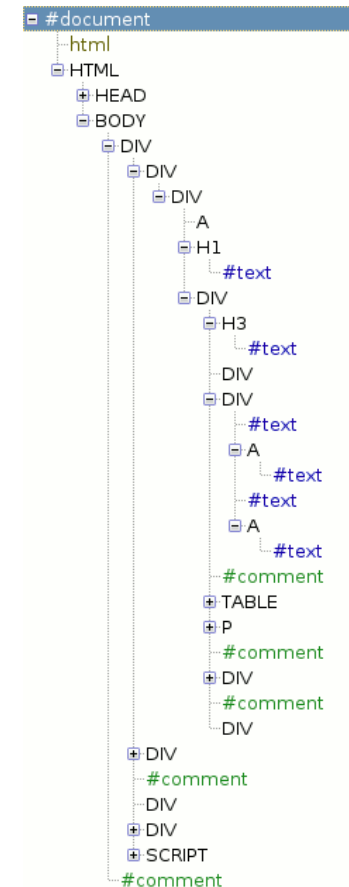
Single-Page Anwendung



Single-Page-Anwendung

- Selektive Aktualisierung von Seite...
- Dynamische Kommunikation mit Web-Server, um selektiven HTML-Content abzurufen (häufig per Rest-Service)
- DOM-Tree wird auf Basis der abgerufenen Daten modifiziert
- Umsetzung häufig über
 - AJAX
 - JavaScript-Frameworks (AngularJS, React, Vue.js)

DOM-Tree



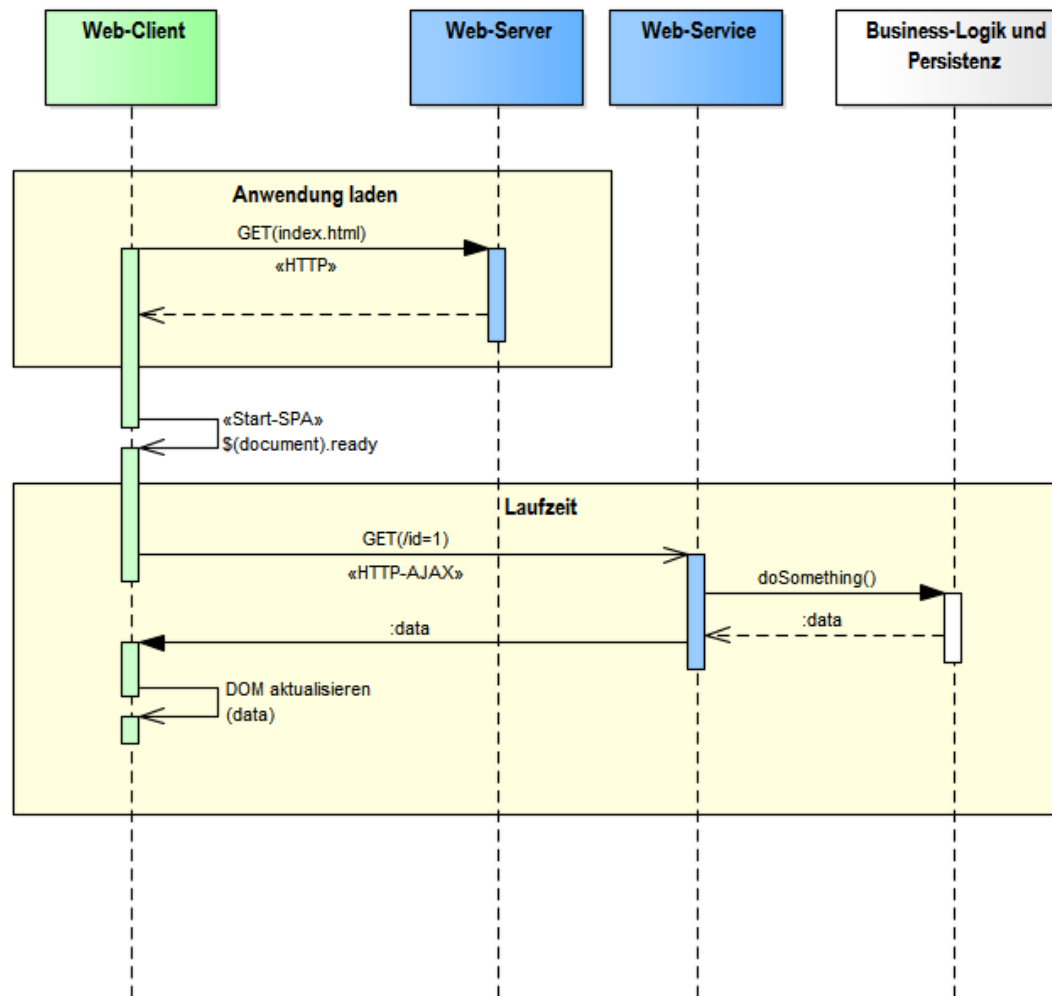
Quelle: Wikipedia

SPA nutzt AJAX

Grundlage für die selektive Aktualisierung der Web-Seite ist AJAX

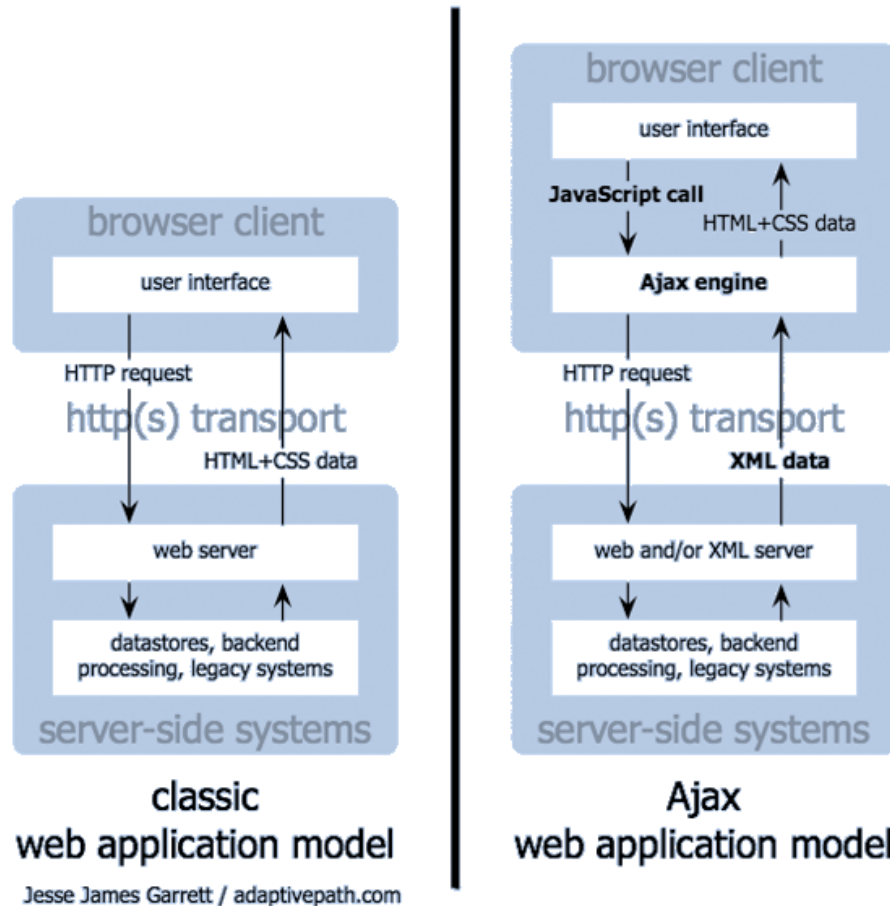
- **Asynchronous JavaScript And XML**
- Schritte
 1. Aktion von User oder Event startet asynchronen Aufruf (blockiert nicht die weitere Ausführung)
 2. Anfrage geht beim Server ein und wird bearbeitet
 3. Server ruft JavaScript Call-Back auf
 4. Daten vom Back-end werden abgerufen
- Antwort vom Back-end häufig als JSON-Dokument
- JavaScript bestimmt Knoten in DOM-Tree, die geändert werden sollen
- Anpassung von Seite im Browser auf Basis der erhaltenen Daten

Kommunikationsverlauf bei SPA



Quelle: Wikipedia

SPA und AJAX



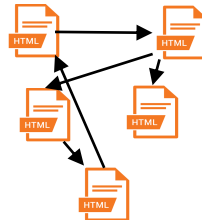
Quelle: adaptivepath.com / abgewandelt durch Ersetzen von XML durch JSON

MPA und SPA: Anwendungssteuerung

Bei Multi-Page-Anwendung ist die Anwendungssteuerung auf dem Server, beim SPA beim Client

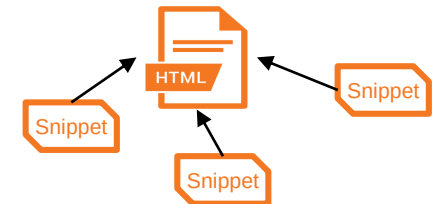
Multi-Page Anwendung

- Umsetzung mit JSP/ ASP/ Servlets, etc.
- Schritte
 1. Interaktion
 2. Statusänderung auf Server
 3. Nächste Seite wird bestimmt
 4. Rendering von Seite und Lieferung an Browser



Single-Page Anwendung

- Schritte
 1. Erste Seite laden
 2. Interaktion
 3. Änderung von „Modell“ (Rest-Call)
 4. Callback an „Presenter“ (Rest-Reponse)
 5. Aktualisierung des betreffenden Teils der Seite

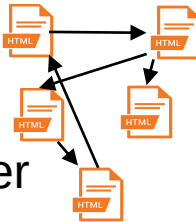


MPA und SPA: Einsatzgebiete

MPA und SPA sind für unterschiedliche Anwendungsbereiche ausgelegt

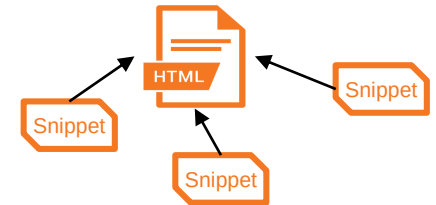
Multi-Page Anwendung

- Komplexe Fachlogik
- Konsistenter Status auf Server von großer Bedeutung
- Nutzung von kompilierten Programmiersprachen
- Enge Kopplung von Front-end und Back-end



Single-Page Anwendung

- Angenehmes Look-and-Feel
- Reduzierter Traffic
- Bessere Interaktivität
- Lose Kopplung von Front-end und Back-end
- Anfällig für Cross-Site-Scripting



SPA: weitere Aspekte

- Ermöglicht länger laufende Client-Interaktionen (z.B. Chat, Video-Stream, etc. per Web-Sockets - bidirektionale Verbindung zwischen einer Webanwendung und einem WebSocket-Server), da kein Page-Relaod
- Der Sitzungszustand wird in der Client-Applikation gespeichert, nicht im Server.
 - „Refresh“ führt zu Neustart der Anwendung
 - Durch die clientseitige Sitzungsverwaltung können serverseitigen Funktionalitäten zustandslos implementiert werden:
 - Erleichtert Parallelisierung!
 - Stelle aber besondere Anforderungen an Back-end
 - Serverseitig werden keine Ressourcen pro aktive Clientsitzung verbraucht
- Serverseite und Client-Seite können unabhängig voneinander implementiert und getestet werden.
- Wiederverwendung von serverseitigen Services (da lose Kopplung)

SPA: Anwendungsgebiete

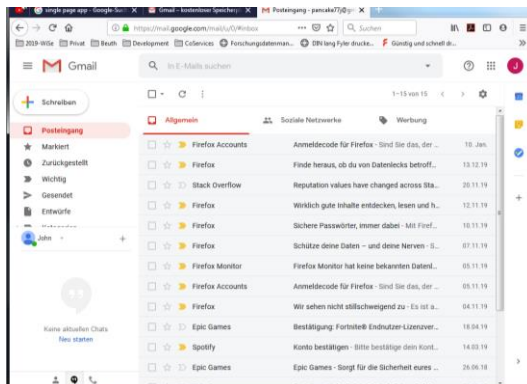
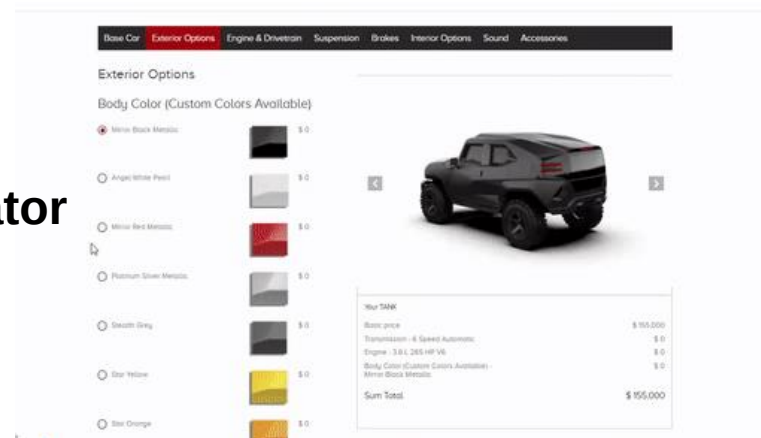
- Beispiele
 - Facebook, Google Gmail, Google Maps und Twitter,
 - In der Regel B2C
- Kleine Projekte
 - Überschaubares Fachkonzept
 - Umfang der clientseitig umgesetzten Geschäftslogik gering
 - Hauptaugenmerk auf der Präsentationsschicht
- Hohe Interaktivität
 - Für Anwendungen wie Computerspiele
 - Hohe Interaktivität und kurze Latenzzeiten
 - Durch HTML5, Canvas, WebSockets und WebGL stellen RIA/JS-SPAs Alternative zu Flash/ MS Silverlight, etc.
- Hybride Web-Apps
 - Einbettung in native mobile Anwendungen (z.B. Apache Cordova oder modifiziert React Native)

Web-Architekturen: SPA Beispiele



Trello

Car Configurator

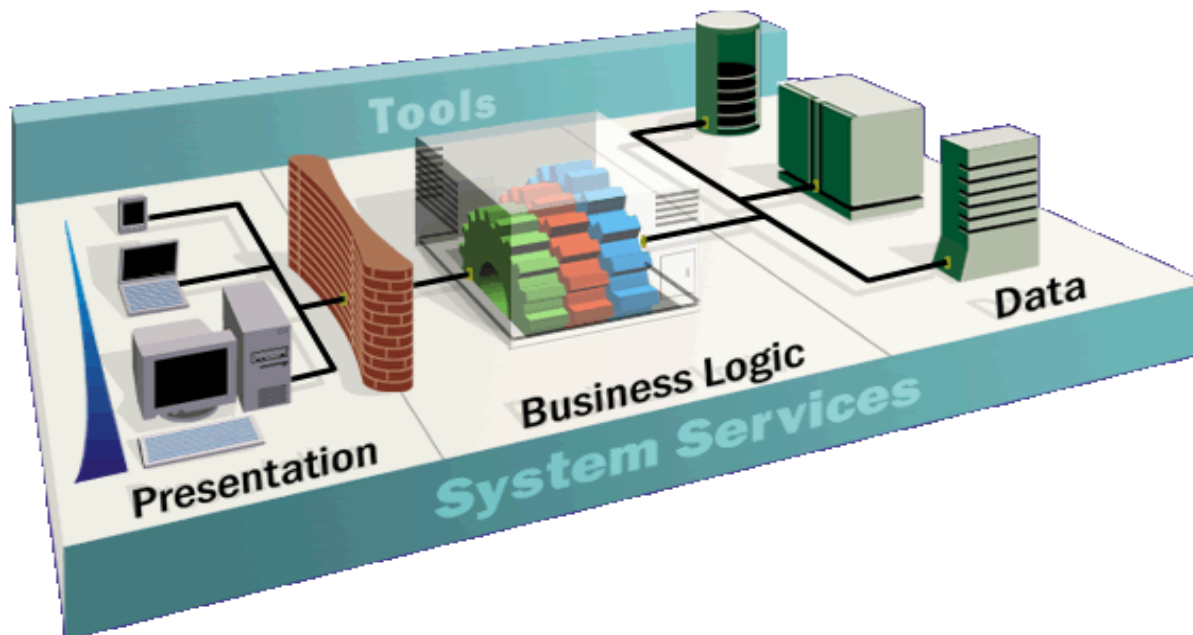


Gmail

Arten von Web-Anwendungen: eBusiness

Web-Architekturen: eBusiness-Architektur

Die IT-Architektur für eBusiness-Systeme zeichnet sich in der Regel durch eine komplex Geschäftslogik und hohe Anforderungen an Ausfallsicherheit, Sicherheit, etc. aus.

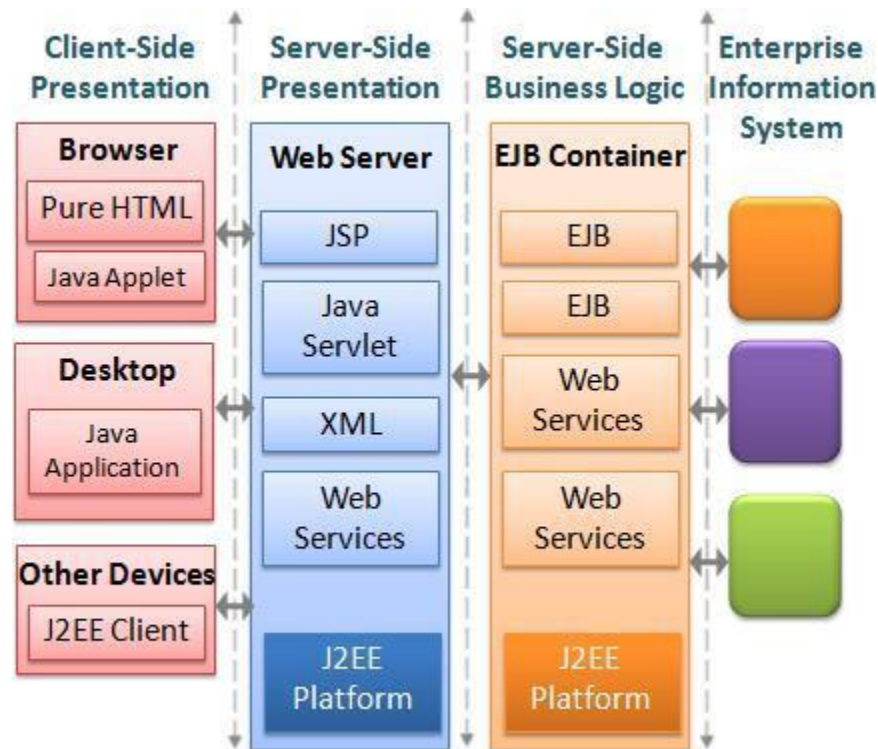


Quelle: unbekannt

eBusiness-Architekturen

Java EE, ist Spezifikation einer Softwarearchitektur für die transaktionsbasierte Ausführung von in Java programmierten Anwendungen und insbesondere Webanwendungen

Quelle: Wikipedia



Beispiel: J2EE

Quelle: <http://www.techiesguide.com/j2ee.html>

Web-Architekturen: J2EE

Komponenten

- Back-end: unterteilt in Presentation/Service-Layer und Business-Logic-Layer
- Presentation-Layer
 - Servlet-Container
 - JSP (Java Server Pages)
 - Web-Services
- Business-Logic-Layer
 - Application Server (Ausfallsicherheit, Transaktionsmanagement, Authentifizierung, Autorisierung, Verzeichnisdienste, Datenbankbindung, etc.)
 - Umsetzung der Business Logik (Kunde, Shopping-Cart, etc.)
 - Session and Entity Beans (EJBs – Enterprise Java Beans)

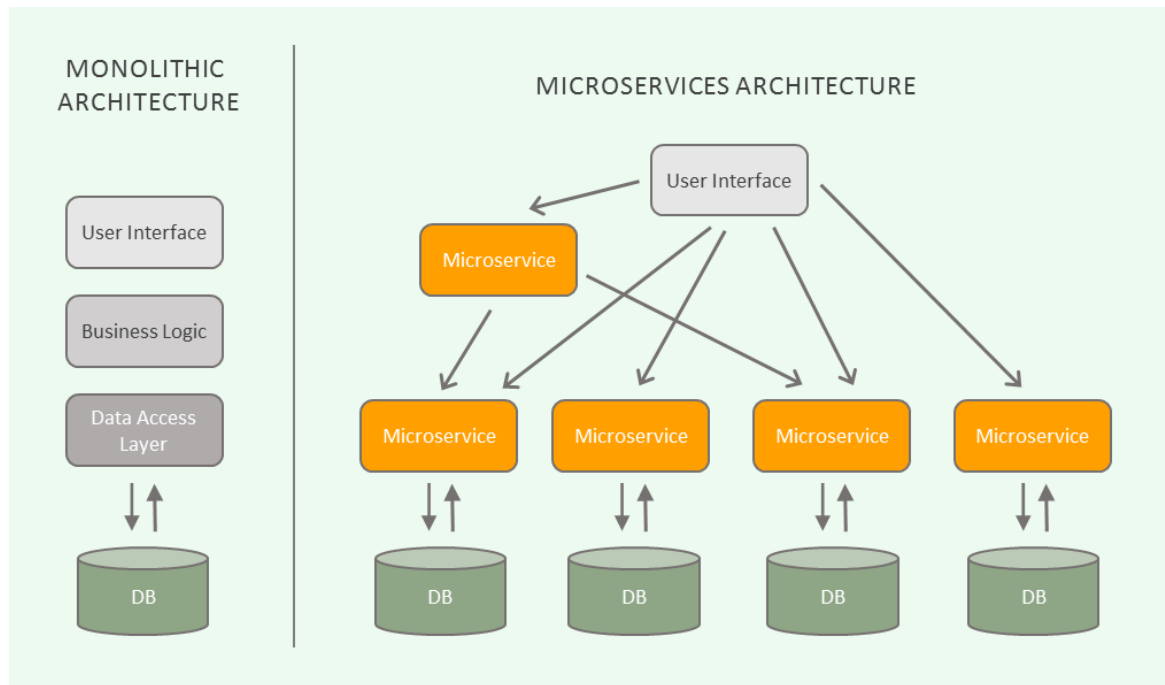
Web-Architekturen: .Net

Komponenten

- Sehr ähnlich zu J2EE
- JSP → ASP.net
- Java-Runtime → CLR (Common Language Runtime)
- Wesentlicher Unterschied:
 - Java/ J2EE: plattformunabhängig (Betriebssystem)
 - .Net: sprachunabhängig (Programmierung in Vb.net, C#, C++, Java, etc.)

Web-Architekturen: Microservice-Architekturen

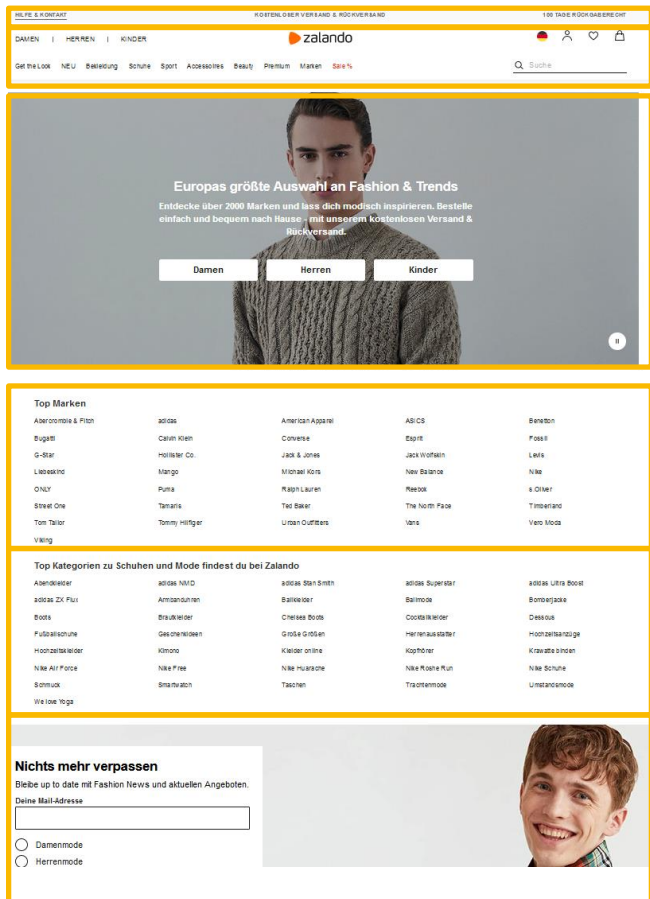
*Umsetzung von Software aus zahlreichen, unabhängigen Prozessen.
Ansatz zur Parallelisierung von Entwicklungsarbeiten*



Quelle: <https://algorithmia.com/blog/introduction-to-microservices>

Web-Architekturen: Microservices

- Beispiele: Amazon, Twitter, eBay, Zalando, etc.



Team 1

Team 2

Team 3

Team 4

Team 5

Team 6

Einzelne Bestandteile
der Web-Seite
werden von
unterschiedlichen
Teams umgesetzt

Quelle: <https://www.zalando.de>

Web-Stacks

Web-Stack

- Konkrete Umsetzung einer Architektur durch Web-Stack
- Web-Application-Solution Stack/ Softwarestack/ Lösungsstack (kurz: Web-Stack)
- Alle Software-Komponenten, die zum Betreiber der Web-Anwendung notwendig sind, sie bilden eine Plattform (Laufzeitumgebung für Softwarekomponenten)
- Bestandteile
 - Betriebssystem
 - Web-Server
 - Laufzeitumgebung
 - Datenpersistenz
 - Programmiersprache
 - DIE
 - Sonstiges

Web-Stack: Beispiele

Web Stack Komponenten	LAMP	.Net	MEAN
 Sonstiges		ASP.Net, WPF...	Angular, Express.js
IDE		Visual Studio	
Skriptsprache	Perl/PHP/Python	C#,F#,VB.Net	JavaScript, TypeScript
Datenpersistenz	MySQL/MariaDB	SQL Server/...	MongoDB
Laufzeitumgebung	Perl/PHP/Python	.NET	Browser
Webserver	Apache	IIS	Node.js
Betriebssystem	Linux	Windows	